# Qwen Sentinel: AI-Powered Computer Lab Surveillance System

A Real-Time Video Analytics Platform Using Vision-Language Models

Technical Report

February 2026

*Built with Qwen Vision-Language Models, OpenCV, Gradio, and PyTorch*

# Contents


**1 Introduction** **3**
1.1 Background and Motivation . . . 3
1.2 Project Overview . . . 3
1.3 Scope . . . 3

**2 System Architecture** **3**
2.1 High-Level Architecture . . . 3
2.2 Module Structure . . . 4
2.3 Threading Model . . . 5

**3 AI Backend** **5**
3.1 Supported Models . . . 5
3.2 Model Loading Pipeline . . . 6
3.3 Inference Pipeline . . . 6
3.4 Analysis Modes . . . 7

**4 Video Capture and Frame Management** **7**
4.1 Input Sources . . . 7
4.2 Sliding Window Frame Buffer . . . 7
4.3 Frame Sampling Strategies . . . 7
4.4 Video File Processing . . . 8

**5 Two-Pass AI Analysis** **8**
5.1 Pass 1: Activity Detection (Multi-Frame) . . . 8
5.2 Pass 2: Bounding Box Localisation (Single-Frame) . . . 8

**6 User Interface** **9**
6.1 Model Loading Panel . . . 9
6.2 Live Context Monitor Tab . . . 10
6.3 Manual Analysis Tab . . . 10

**7 Alert Handling and Evidence Collection** **10**
7.1 Alert Lifecycle . . . 10
7.2 Evidence Snapshots . . . 11
7.3 Debug Contact Sheets . . . 11

**8 Configuration and Tuning** **11**
8.1 System Parameters . . . 11
8.2 Runtime Adjustability . . . 11

**9 Technology Stack** **12**

**10 Installation and Deployment** **12**
10.1 Prerequisites . . . 12
10.2 Installation Steps . . . 12
10.3 Running the Application . . . 13

**11 Design Decisions and Trade-offs** **13**

11.1 Why Vision-Language Models? . . . . . 13
11.2 Two-Pass vs. Single-Pass Analysis . . . . . 13
11.3 Thread Safety . . . . . 13
11.4 Quantisation Trade-off . . . . . 14

**12 Limitations and Future Work** **14**
12.1 Current Limitations . . . . . 14
12.2 Future Enhancements . . . . . 14

**13 Conclusion** **14**

**A Default Analysis Prompt** **15**

**B Strict Mode Suffix (Video Files)** **15**

**C Bounding Box Localisation Prompt** **15**

# 1 Introduction

## 1.1 Background and Motivation

Computer laboratories in educational institutions house expensive equipment and serve as shared workspaces for students and staff. Ensuring that lab rules are followed—such as prohibiting eating, drinking, running, fighting, sleeping, phone usage, and equipment tampering—traditionally requires dedicated human supervision. This is labour-intensive, error-prone, and does not scale.

Recent advances in **Vision-Language Models (VLMs)** have made it possible to analyse visual data with natural-language understanding. These models can interpret video frames and answer questions about observed activities, opening the door to intelligent, automated surveillance systems that go beyond simple motion detection.

## 1.2 Project Overview

This project, titled **Qwen Sentinel**, implements a real-time AI-powered surveillance system for computer laboratories. The system:

- Captures video from **live cameras**, **RTSP/HTTP streams**, or **uploaded video files**.
- Maintains a **sliding window frame buffer** and periodically samples frames using configurable strategies.
- Sends sampled frames to a **Qwen Vision-Language Model** for activity detection via natural-language prompting.
- Performs a **two-pass analysis**: first detecting prohibited activities, then localising the offender with bounding-box regression.
- Provides a **web-based dashboard** (Gradio) with live feed display, alert overlays, event logging, and debug tools.
- Saves **evidence snapshots** with annotated bounding boxes for review.

## 1.3 Scope

The system targets the following prohibited activities in a lab environment:

1. Running (fast movement across frames)
2. Fighting or aggressive physical contact
3. Eating or drinking
4. Tampering with equipment
5. Sleeping
6. Phone usage

# 2 System Architecture

## 2.1 High-Level Architecture

The system follows a **multi-threaded producer–consumer architecture** with three main layers:

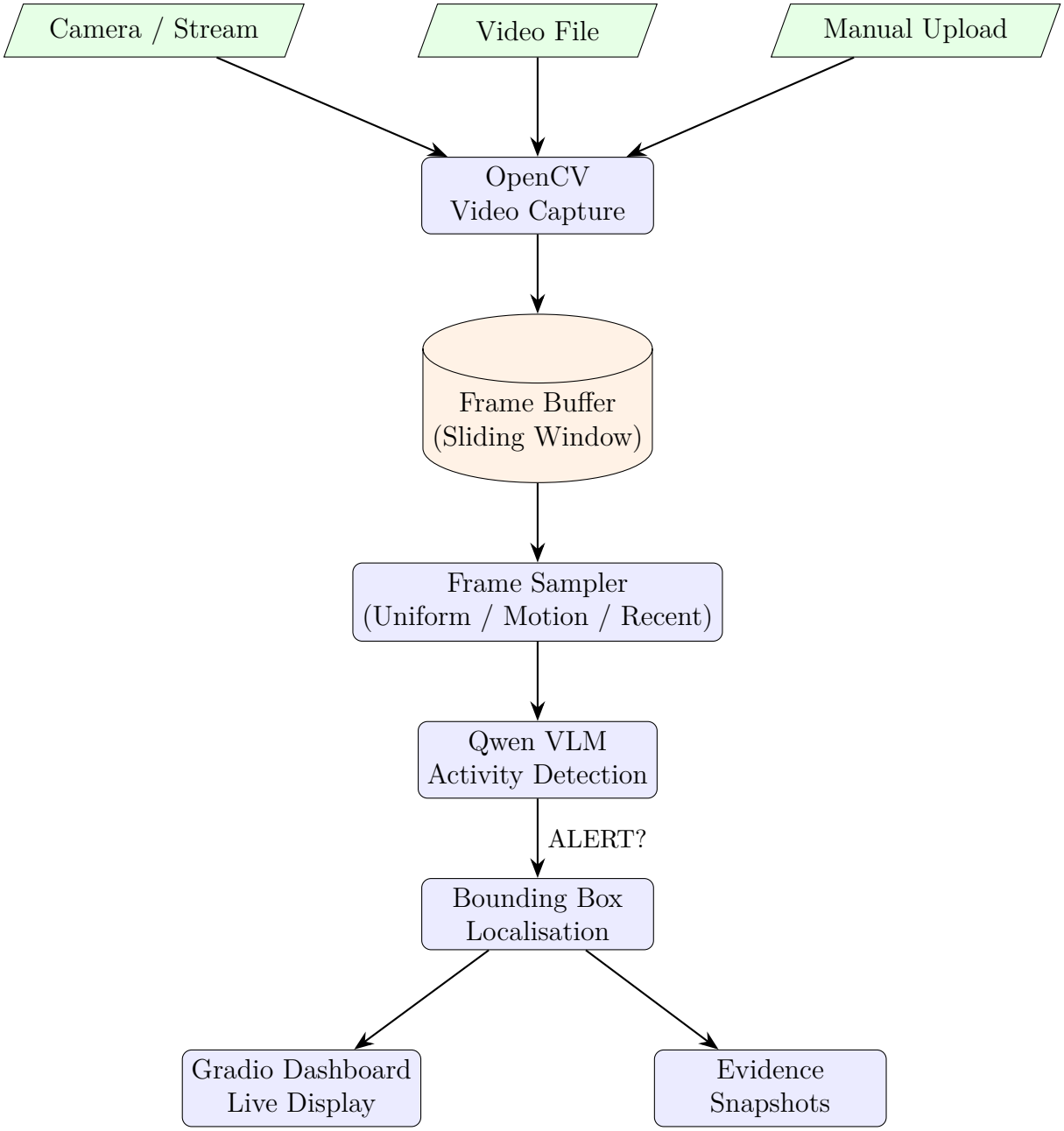


Figure 1: High-level system architecture of Qwen Sentinel.

## 2.2 Module Structure

The codebase is organised into two main Python modules:

Table 1: Project file structure.

| File / Directory | Description |
|---|---|
| `src/app.py` | Main application: UI, video capture, frame sampling, AI orchestration, display rendering, event logging (1107 lines). |
| `src/backend.py` | AI backend: model loading, inference pipeline using HuggingFace Transformers and Qwen VL utilities (292 lines). |
| `requirements.txt` | Python dependencies (77 packages). |
| `install.ps1` | PowerShell installation script for PyTorch (CUDA), Transformers, and optional GGUF/llama-cpp-python support. |
| `evidence_snapshots/` | Directory for saved alert evidence images. |

### 2.3 Threading Model

The system uses a **multi-threaded design** to decouple frame capture from AI inference:

- **Main Thread (Gradio Generator):** Captures frames from the video source, appends them to a shared frame buffer, renders display frames with overlays, and yields UI updates to the Gradio frontend.
- **AI Worker Thread:** A long-lived daemon thread that waits on a `threading.Event`. When triggered by the main thread (based on a configurable time interval), it takes a snapshot of the frame buffer and runs AI analysis. Results are written to shared state under a lock.
- **Synchronisation:** Three locks protect shared data—`buffer_lock` for the frame deque, `state_lock` for AI results, and `log_lock` for log histories.

## 3 AI Backend

### 3.1 Supported Models

The system supports multiple **Qwen Vision-Language Models** from the Qwen family, loaded via HuggingFace Transformers:

Table 2: Supported Qwen VL models.

| Model Name | HuggingFace Model ID |
|---|---|
| Qwen2-VL-2B-Instruct | `Qwen/Qwen2-VL-2B-Instruct` |
| Qwen2-VL-7B-Instruct | `Qwen/Qwen2-VL-7B-Instruct` |
| Qwen2.5-VL-3B-Instruct | `Qwen/Qwen2.5-VL-3B-Instruct` |
| Qwen2.5-VL-7B-Instruct | `Qwen/Qwen2.5-VL-7B-Instruct` |
| Qwen3-VL-2B-Instruct | `Qwen/Qwen3-VL-2B-Instruct` |
| Qwen3-VL-4B-Instruct | `Qwen/Qwen3-VL-4B-Instruct` |
| Qwen3-VL-8B-Instruct | `Qwen/Qwen3-VL-8B-Instruct` |
| FP8 variants | FP8-quantised versions of Qwen3-VL models |

## 3.2 Model Loading Pipeline

The model loading process in `backend.py` performs the following steps:

1. **Cache Configuration:** The HuggingFace cache directory is configured to reside within the project directory (`.hf_cache/`) to keep downloads self-contained. Environment variables `HF_HOME`, `HF_HUB_CACHE`, and `TRANSFORMERS_CACHE` are set accordingly.
2. **Cleanup:** If a model is already loaded, the previous model and processor are deleted, and GPU memory is freed via `gc.collect()` and `torch.cuda.empty_cache()`.
3. **Quantisation (Optional):** If 4-bit quantisation is enabled, a `BitsAndBytesConfig` is created with NF4 quantisation type and `bfloat16` compute dtype. This reduces VRAM usage significantly.
4. **Processor Loading:** The model's tokenizer and image processor are loaded via `AutoProcessor.from_pretrained()`.
5. **Model Loading:** The model itself is loaded via `AutoModelForImageTextToText.from_pretrain` with automatic device mapping (`device_map="auto"`) and automatic dtype selection.

## 3.3 Inference Pipeline

The inference pipeline in `_run_inference()` follows the standard Qwen VL pattern:

1. **Chat Template Application:** The conversation messages (system prompt + user content with images/video) are formatted using the processor's chat template with `add_generation_prompt=True`. For Qwen3 models, chain-of-thought reasoning is disabled (`enable_thinking=False`).
2. **Vision Info Processing:** The `qwen_vl_utils.process_vision_info()` utility extracts and preprocesses image and video tensors from the message payload.
3. **Tokenisation:** Text, images, and (optionally) video tensors are fed through the processor to produce input tensors.
4. **Generation:** `model.generate()` produces output token IDs with a configurable maximum of 256 new tokens.

5. **Decoding:** Output tokens are trimmed (removing the input portion) and decoded to text. For Qwen3 models, any `<think>...</think>` reasoning blocks are stripped from the output.

### 3.4 Analysis Modes

The backend exposes three analysis methods:

**`analyze_video_clip()`** Sends a list of PIL frames as a sequence of `image` content blocks. This is the primary analysis mode used for camera streams.

**`analyze_video_clip_as_video()`** Saves frames to a temporary directory and sends them as a `video` content block with temporal FPS metadata. Falls back to `analyze_video_clip()` if video processing fails. Used optionally for video file analysis.

**`analyze_single_image()`** Sends a single PIL image for analysis. Used for bounding-box localisation and manual single-image uploads.

## 4 Video Capture and Frame Management

### 4.1 Input Sources

The system supports three types of video input:

1. **Local Webcam:** Specified by camera index (e.g., `0` for the default camera). Opened via `cv2.VideoCapture(int)`.
2. **Network Stream:** RTSP or HTTP URLs are passed directly to `cv2.VideoCapture(str)`, enabling integration with IP cameras and streaming servers.
3. **Video File:** Uploaded through the Gradio interface. The file is copied to a temporary location to avoid file-lock contention on Windows, with a retry mechanism (`safe_copy_video`).

### 4.2 Sliding Window Frame Buffer

For live camera/stream sources, frames are stored in a **thread-safe sliding window buffer** implemented as a `collections.deque` with a configurable maximum length:

$$\text{maxlen} = \text{buffer_seconds} \times \text{fps}$$

The default buffer holds 4 seconds of video at the source's frame rate. This ensures the AI always analyses the most recent footage while bounding memory usage.

### 4.3 Frame Sampling Strategies

Before sending frames to the AI model, a subset is sampled from the buffer. Three strategies are available:

**Uniform Sampling** Selects frames at evenly spaced intervals across the buffer using `numpy.linspace`. Provides consistent temporal coverage.

**Recent Focus** Concentrates sampling on the most recent 60% of the buffer while still including earlier frames. Useful when the most recent activity is most relevant.

**Motion Focus** Computes inter-frame differences (using downscaled $64 \times 36$ grayscale images and `cv2.absdiff`) to identify frames with the highest motion scores. Selects the top-$k$ frames by motion magnitude, back-filling with uniform indices if needed. This strategy is particularly effective for detecting transient prohibited activities.

Each sampled frame is resized to a height of 480 pixels (configurable) and converted from BGR to RGB as a PIL `Image`.

### 4.4 Video File Processing

Video files use a different pipeline than live streams:

- Instead of real-time buffering, the video is read at maximum speed and split into **segments** of length buffer_seconds $\times$ fps frames.
- Each segment is analysed sequentially by the AI, producing per-segment results.
- A **Strict Mode** prompt suffix is appended for video files, instructing the AI to err on the side of alerting rather than defaulting to "Safe" when uncertain.
- Progress is reported per segment, and the user sees a preview frame during analysis.

## 5 Two-Pass AI Analysis

The core analysis follows a **two-pass approach**:

### 5.1 Pass 1: Activity Detection (Multi-Frame)

A set of sampled frames (typically 8–16) is sent to the Qwen VLM with a detailed natural-language prompt. The prompt instructs the model to:

- Analyse all visible areas, including backgrounds and partially obscured regions.
- Use temporal context across frames to detect motion-dependent activities (running, fighting).
- Output either `Status: Safe` or `ALERT: [Description of activity]`.

The default prompt is:

> *"Analyze this video clip from a computer lab surveillance camera. Look at all the shown places and people in the clip. Even in the background and obscured areas. People may be partially visible, far away, or in groups. Use the full clip context to understand their actions. The clip may show multiple people. Determine if any PROHIBITED activities are occurring..."*

### 5.2 Pass 2: Bounding Box Localisation (Single-Frame)

If Pass 1 returns an `ALERT`, a second inference pass is run on the **last frame at higher resolution** (up to 720p) with a dedicated localisation prompt:

> *"This surveillance image contains a person performing a prohibited activity: {activity}. Locate the person performing that activity. Return ONLY the bounding box as [xmin, ymin, xmax, ymax] on a 0–1000 scale..."*

The bounding box coordinates are parsed from the model output using regular expressions, normalised to $[0, 1]$ scale, and used to draw red rectangles with activity labels on the display frame and evidence snapshot.

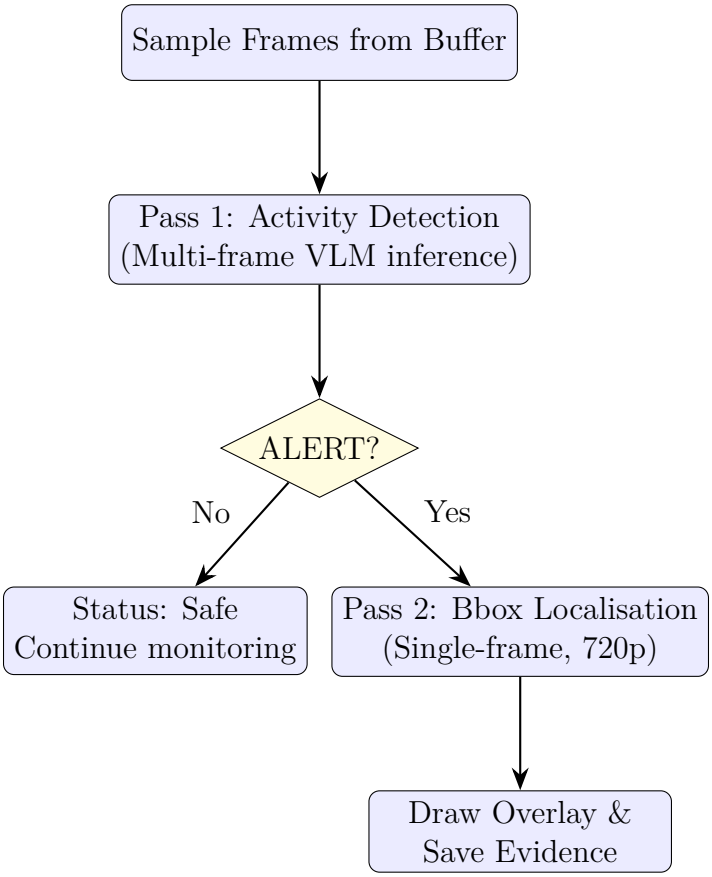


Figure 2: Two-pass AI analysis pipeline.

# 6 User Interface

The web-based user interface is built with **Gradio** and is accessible from any browser on the network.

## 6.1 Model Loading Panel

At the top of the interface, a panel allows the user to:

- Select a Qwen VL model from a dropdown list.
- Toggle 4-bit quantisation to reduce VRAM requirements.
- Click "Load Model" to download (on first use) and initialise the model.
- View the loading status.

Models are downloaded at runtime from HuggingFace Hub and cached locally.

### 6.2 Live Context Monitor Tab

The primary monitoring tab provides:

- **Live Video Feed:** A streaming image display with a status bar overlay showing the latest AI detection result. Safe status is shown in green; alerts in red.
- **Source Selection:** Radio buttons to choose between Camera and Video File input.
- **Configuration Controls:**
  - Camera ID or stream URL text field.
  - Video file upload widget (MP4, AVI, MOV, MKV).
  - AI check interval slider (1–10 seconds).
  - Sampling strategy dropdown (Uniform, Recent Focus, Motion Focus).
  - Buffer/segment length slider (2–30 seconds).
  - Frames per segment slider (4–32 frames).
  - Video mode toggle (experimental temporal encoding).
- **Prompt Editor:** An expandable text area for customising the AI analysis prompt.
- **Start/Stop Controls:** Buttons to begin and halt monitoring.
- **Event Log:** A scrollable text area showing timestamped detection results.
- **Debug Log:** Detailed sampling and inference diagnostics (buffer sizes, frame hashes, model outputs).
- **Debug Options:** Checkboxes to enable debug logging and to save sampled frame contact sheets.

### 6.3 Manual Analysis Tab

A secondary tab for one-shot analysis of uploaded images or videos:

- Upload a single image or video file.
- Optionally customise the analysis prompt.
- Click "Analyze" to receive the AI's assessment.
- Results are displayed as Markdown text alongside the uploaded image.

## 7 Alert Handling and Evidence Collection

### 7.1 Alert Lifecycle

When the AI detects a prohibited activity:

1. The result text (e.g., `ALERT: Person eating at desk`) is stored in shared state.
2. The **alert linger timer** keeps the alert overlay visible for check_interval + 1.5 seconds, ensuring the user sees it even between AI analysis cycles.
3. Bounding boxes from Pass 2 are drawn on the live feed as red rectangles with the activity description as a label.
4. The alert is logged in the Event Log with a timestamp and alarm emoji.

### 7.2 Evidence Snapshots

For every alert, the system automatically saves an **evidence snapshot**:

- The last frame from the analysed buffer segment is copied.
- Bounding boxes and activity labels are burned into the image using OpenCV drawing functions.
- The file is saved to `evidence_snapshots/` with a filename pattern: `alert_YYYYMMDD_HHMMSS_<act`
- Activity descriptions are sanitised for filesystem safety.

### 7.3 Debug Contact Sheets

When debug frame saving is enabled, sampled frames are stitched into a **contact sheet** (grid of thumbnails) and saved as a JPEG. This allows visual verification of which frames the AI model received, aiding in prompt and sampling strategy tuning.

## 8 Configuration and Tuning

### 8.1 System Parameters

The `Config` dataclass centralises all tunable parameters:

Table 3: Configurable system parameters.

| Parameter | Default | Description |
|---|---|---|
| `buffer_seconds` | 4 | Duration of the sliding window buffer in seconds. |
| `sample_frames` | 8 | Number of frames sampled per AI analysis. |
| `check_interval` | 2.0 | Seconds between AI analysis triggers. |
| `resize_height` | 480 | Height (px) to which sampled frames are resized. |
| `display_max_width` | 1024 | Maximum display width for the live feed. |
| `display_frame_interval` | 4 | Yield a display frame every $n$ captured frames. |
| `alert_linger_extra` | 1.5 | Extra seconds to keep alert overlays visible. |
| `max_log_entries` | 100 | Maximum event log entries retained. |
| `max_debug_entries` | 200 | Maximum debug log entries retained. |

### 8.2 Runtime Adjustability

The Gradio UI exposes several parameters for **runtime adjustment** without restarting the application:

- Buffer/segment length (2–30 seconds).
- Number of sampled frames per segment (4–32).
- AI check interval (1–10 seconds).
- Sampling strategy (Uniform, Recent Focus, Motion Focus).
- Analysis prompt (fully editable).
- Video mode toggle for temporal encoding.
- Debug logging and frame saving toggles.

# 9 Technology Stack

Table 4: Core technology stack.

| **Technology** | **Role** |
|---|---|
| Python 3.13/3.14 | Primary programming language. |
| PyTorch (CUDA 13.0) | Deep learning framework with GPU acceleration. |
| HuggingFace Transformers | Model loading, tokenisation, and generation. |
| Qwen-VL-Utils | Vision-language preprocessing utilities for Qwen models. |
| BitsAndBytes | 4-bit NF4 quantisation for reduced VRAM usage. |
| OpenCV | Video capture, frame manipulation, drawing overlays. |
| Gradio | Web-based UI framework with streaming support. |
| Pillow (PIL) | Image format conversion and manipulation. |
| NumPy | Numerical operations, frame sampling, motion scoring. |
| MoviePy / ImageIO | Video I/O utilities. |

# 10 Installation and Deployment

## 10.1 Prerequisites

- Python 3.13 or 3.14.
- NVIDIA GPU with CUDA support (recommended; CPU fallback available).
- Windows operating system (PowerShell install script provided).
- A Python virtual environment (`venv`).

## 10.2 Installation Steps

The provided `install.ps1` PowerShell script automates the setup:

1. Activates the virtual environment (or detects an existing active environment).
2. Installs PyTorch and TorchVision with CUDA 13.0 support from the PyTorch wheel index.

3. Installs the latest HuggingFace Transformers from the Git repository.
4. Optionally installs `llama-cpp-python` with CUDA support for GGUF model inference (experimental). Falls back to CPU build if the CUDA build fails.
5. Installs all remaining dependencies from `requirements.txt`.
6. Removes the `hf-xet` package to avoid model download issues.

### 10.3 Running the Application

After installation, the application is launched with:

```
python src/app.py
```

The Gradio interface starts on `http://0.0.0.0:7860` and is accessible from any device on the network.

## 11 Design Decisions and Trade-offs

### 11.1 Why Vision-Language Models?

Traditional computer vision approaches (object detection, pose estimation, action recognition) require task-specific training data and separate models for each activity. VLMs offer:

- **Zero-shot generalisation:** The model understands activities described in natural language without task-specific fine-tuning.
- **Prompt flexibility:** The set of prohibited activities can be changed instantly by editing the text prompt.
- **Contextual reasoning:** The model can reason about multi-frame temporal context and spatial relationships.

### 11.2 Two-Pass vs. Single-Pass Analysis

A single-pass approach that simultaneously detects activities and localises offenders was found to be unreliable. Splitting detection (multi-frame, lower resolution) from localisation (single-frame, higher resolution) improves both accuracy and specificity:

- Pass 1 benefits from **temporal context** across multiple frames.
- Pass 2 benefits from **higher spatial resolution** on a single frame for precise bounding-box regression.

### 11.3 Thread Safety

The producer–consumer design with explicit locks ensures that:

- Frame capture runs at full speed without being blocked by AI inference.
- AI inference can take variable time without dropping frames.
- The UI remains responsive during long inference cycles.

### 11.4 Quantisation Trade-off

4-bit NF4 quantisation via BitsAndBytes reduces VRAM usage by approximately 4× (e.g., a 7B model fits in ∼6 GB instead of ∼24 GB). The trade-off is a small reduction in model accuracy, which is generally acceptable for surveillance activity classification.

## 12 Limitations and Future Work

### 12.1 Current Limitations

- **Inference Latency:** VLM inference takes 1–5 seconds per analysis cycle depending on the model size and GPU. This introduces a delay between the occurrence of a prohibited activity and its detection.
- **GGUF Support:** GGUF model format support via `llama-cpp-python` is work-in-progress and requires additional build tools (CMake, Visual Studio Build Tools, CUDA Toolkit).
- **False Positives:** VLMs can occasionally misinterpret benign activities (e.g., drinking water from a bottle vs. reaching for a mouse). Prompt engineering helps but does not eliminate this.
- **Single Camera:** The current architecture monitors one video source at a time.
- **Limited Testing:** The manual upload analysis tab is noted as not thoroughly tested.

### 12.2 Future Enhancements

- **Multi-camera support** with a unified dashboard.
- **Persistent alert database** with search and filtering.
- **Fine-tuned models** trained on lab surveillance data for improved accuracy.
- **Integration with notification systems** (email, SMS, push notifications).
- **GGUF model support** for CPU-only deployments.
- **Edge deployment** optimisation for embedded devices.

## 13 Conclusion

The **Qwen Sentinel** system demonstrates a practical application of Vision-Language Models for automated computer lab surveillance. By combining a sliding-window frame buffer with configurable sampling strategies and a two-pass AI analysis pipeline, the system achieves real-time detection and localisation of prohibited activities without any task-specific training. The Gradio-based web interface provides an accessible dashboard for monitoring, configuration, and evidence review. While inference latency and occasional false positives remain challenges, the prompt-driven approach offers unprecedented flexibility in defining and updating the set of monitored activities. The system serves as

a foundation for intelligent surveillance solutions that can be extended to multi-camera deployments and integrated with institutional security infrastructure.

# A Default Analysis Prompt

```
Analyze this video clip from a computer lab surveillance camera.
Look at all the shown places and people in the clip. Even in the
background and obscured areas. People may be partially visible,
far away, or in groups. Use the full clip context to understand
their actions. The clip may show multiple people. Determine if
any PROHIBITED activities are occurring.

Check for PROHIBITED activities that require motion context:
1. Running (moving fast across frames)
2. Fighting or Aggressive Physical Contact
3. Eating/Drinking
4. Tampering with equipment
5. Sleeping
6. Phone usage

OUTPUT FORMAT:
If SAFE: 'Status: Safe'
If DANGEROUS: 'ALERT: [Description of activity]'
```

# B Strict Mode Suffix (Video Files)

```
STRICT MODE (video file):
If ANY prohibited activity appears in ANY frame, output ALERT.
Do not default to SAFE if uncertain. If motion suggests running
or aggressive contact, alert. If a person appears to eat/drink,
tamper with equipment, phone usage, or sleep, alert. Return
exactly one line in the required format.
```

# C Bounding Box Localisation Prompt

```
This surveillance image contains a person performing a prohibited
activity: {activity}. Look carefully at the ENTIRE image. Locate
the person performing that activity. Return ONLY the bounding box
of that person as [xmin, ymin, xmax, ymax] on a 0-1000 scale
where (0,0) is top-left and (1000,1000) is bottom-right. Return
nothing else.
```